

RISC-V ELF psABI - Qualcomm Extensions

Version 0.2, 2025-06-12: Development

Table of Contents

Document Status	1
List of tables	2
Copyright and license information	3
Contributors	4
1. Introduction	5
2. Qualcomm Extensions	6
2.1. ELF File Header	6
2.2. Relocations	6
2.2.1. Calculation Symbols	7
2.2.2. Field Symbols	7
2.3. Code Sequences	7
2.3.1. Absolute Addresses	7
2.3.2. Procedure Calls	8
2.3.3. PC-Relative Branches	8
2.4. Attributes	8
2.5. Additional Linker Relaxation Types	8
2.5.1. Standard to Extended Function/Tail Call Relaxation	9
2.5.2. Extended to Standard Function/Tail Call Relaxation	10
2.5.3. Extended to Compressed Function/Tail Call Relaxation	11
2.5.4. Standard to Extended Global-pointer Relaxation	12
2.5.5. Standard to Extended Global-pointer Address Relaxation	13
2.5.6. Extended to Standard Global-pointer Address Relaxation	14
2.5.7. Standard to Extended Zero-page Load/Store Relaxation	15
2.5.8. Standard to Extended Address Relaxation	16
2.5.9. Zero-page Address Relaxation	17
2.5.10. Extended to Standard Address Relaxation	18
2.5.11. Extended to Compressed Address Relaxation	19
Appendix A: Commentary	21
A.1. Why Have We Not Reused Standard Relocations?	21
A.2. Why Is There Not A Version of the <code>R_RISCV_QC_E_CALL_PLT</code> Relocation Without The PLT?	21
A.3. Why Have We Not Defined a New Code Model?	21
A.4. Why Are There No Relocations for the 48-bit Load and Store Instructions?	22
A.5. Why Do Some Relaxations Require Adjacent Instructions?	23
Bibliography	25

Document Status



This document is in the *Development state*

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

List of tables

- [Table 1.](#) Additional relocation types
- [Table 2.](#) Additional variables used in relocation fields

Copyright and license information

Copyright 2024-2025, Qualcomm Technologies, Inc. All rights reserved.

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY-4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Contributors

This RISC-V specification has been contributed to by:

- Sam Elliott <aelliott@qti.qualcomm.com>

We express our gratitude to everyone that contributed to, reviewed or improved this specification through their comments and questions.

Chapter 1. Introduction

The RISC-V ELF psABI ([Cheng & Clarke, 2022](#)) makes specific allocations for non-standard extensions. This document specifies how Qualcomm is using this non-standard extension space for Qualcomm's non-standard (but public) RISC-V ISA extensions.

Currently, this document includes ABI extensions that cover the following of Qualcomm's non-standard RISC-V ISA extension specifications:

- **Xqci** v0.13.0 ([Hower & Yosher, 2025](#)), which includes:

- **Xqcia** v0.7.0
- **Xqciac** v0.3.0
- **Xqcibi** v0.2.0
- **Xqcibm** v0.8.0
- **Xqcicli** v0.3.0
- **Xqcicm** v0.2.0
- **Xqcics** v0.2.0
- **Xqcicsr** v0.4.0
- **Xqciint** v0.10.0
- **Xqciio** v0.1.0
- **Xqcilb** v0.2.0
- **Xqcili** v0.2.0
- **Xqcilia** v0.2.0
- **Xqcilo** v0.3.0
- **Xqcilsm** v0.6.0
- **Xqcisim** v0.2.0
- **Xqcisls** v0.2.0
- **Xqcisync** v0.3.0



We have taken a few specific directions for this psABI Extension.

*The **Xqci** extension is broadly only defined for the RV32 base ISA, so this ABI is designed for the same. We have not considered 64-bit RISC-V implementations at this time.*

*Together with the **Xqci** extension designers, we settled on using **e.** to denote 48-bit instructions (you could read this as "extended"), much as the standard ISA uses **c.** to denote 16-bit instructions (read as "compressed"). We follow this through when denoting instruction types which start with QC.E, and including "E_" in the relevant vendor relocation names for our 48-bit instructions.*

*We have made no additions to Dynamic relocations as **Xqci** does not add any new whole-program features, just new instructions.*

Additional commentary is available in [Appendix A](#).

Chapter 2. Qualcomm Extensions

The following section makes additions to the tables in the RISC-V ELF psABI ([Cheng & Clarke, 2022](#)). The descriptions below are given only where the Qualcomm Extensions define fields or values reserved in that specification for nonstandard use. Encoding and enumeration spaces reserved for use by standard extensions are denoted as *Standard* in this specification.

2.1. ELF File Header

This section extends the "ELF File Header" section in the RISC-V ELF psABI specification.

e_ident

The Qualcomm Extensions may only be used where the value of `e_ident.EI_CLASS` is `ELFCLASS32`, in addition to the restrictions already present in the RISC-V ELF psABI. This restrictions may be relaxed in future.

e_flags

This field allocates 8 bits for non-standard extensions. This psABI Extension does not allocate any of these bits.



This does not automatically make this psABI Extension compatible with other non-standard psABI Extensions.

2.2. Relocations

This section extends the "Relocations" section in the RISC-V ELF psABI specification.

Qualcomm uses the `QUALCOMM` symbol name with `R_RISCV_VENDOR` relocations ([Müllner & Pazos, 2024](#)), which must precede any relocation code below.

All ELF relocation code names defined by this specification have the prefix `R_RISCV_QC_`.



This table should be read as an addition to the "Relocation types" table in the RISC-V ELF psABI.

Table 1. Additional relocation types

Enum	ELF Reloc Type	Type	Field / Calculation	Description
0-191	<i>Standard</i>	<i>Varies</i>	<i>Varies</i>	Defined in the RISC-V ELF psABI
192	ABS20_U	Static	<i>U-Type</i> S + A	(Signed) 20-bit Absolute address
193	E_BRANCH	Static	<i>QC.EB-Type</i> S + A - P	12-bit PC-relative branch offset (48-bit instruction)
194	E_32	Static	<i>QC.EAI-Type</i> S + A	32-bit Absolute address
195	E_CALL_PLT	Static	<i>QC.EJ-Type</i> S + A - P	32-bit PC-relative function call (via PLT)
196-255	Reserved			Reserved for future use in the Qualcomm Extensions.



There are no relocations for the **Xqci** loads and stores. This is because they cannot materialise a 32-bit address on their own, and any code sequence including these instructions that could materialise a 32-bit address would be longer than the equivalent sequence that uses an **lui** and a load or store with a 12-bit immediate. If we defined 26-bit absolute address relocations, but these would only apply when **rs1** was **zero**, which isn't very helpful. We describe relaxations for **Xqci** loads and stores, below, but these do not require defined relocation types.

2.2.1. Calculation Symbols

Calculation symbols should be interpreted exactly as they are defined in the RISC-V ELF psABI.

2.2.2. Field Symbols



This table should be read as an addition to the "Variables used in relocation fields" table in the RISC-V ELF psABI.

Table 2. Additional variables used in relocation fields

Field	Description
QC.EAI-Type	Specifies the immediate field in a QC.EAI-type instruction
QC.EB-Type	Specifies the immediate field used for the branch target offset in an QC.EB-type instruction
QC.EJ-Type	Specifies the immediate field used for the jump target offset in an QC.EJ-type instruction
QC.EI-Type	Specifies the immediate field in an QC.EI-type instruction
QC.ES-Type	Specifies the immediate field in an QC.ES-type instruction

The new instruction types starting QC are defined in the **Xqci** specification ([Hower & Yosher, 2025](#)).

2.3. Code Sequences

In addition to the code sequences documented in the RISC-V ELF psABI, the following sequences may also be used by a compiler following this extension, when the corresponding architecture extensions have been enabled.



Relocations not used in these code sequences are designed to be used from assembly files, rather than by compilers, as some have restrictions on the addresses that can be materialized.

2.3.1. Absolute Addresses

32-bit absolute addresses in position dependent code can be loaded with a **qc.e.li** instruction (part of **Xqcili**) and the corresponding relocation **R_RISCV_QC_E_32**.

```
qc.e.li a0, symbol+addend    # R_RISCV_VENDOR QUALCOMM
                             # R_RISCV_QC_E_32 symbol, addend
```



This sequence is only 6 bytes (as opposed to an **lui/addi** pair which is 8 bytes). The size overhead of the pair can be reduced in some circumstances, such as when the **addi** is folded into a load or store instruction or when the **lui** is shared between multiple materialisations, so this is not always beneficial.

20-bit absolute addresses in position dependent code can be loaded with a `qc.li` instruction (part of `Xqcili`) and the corresponding relocation `R_RISCV_QC_ABS20_U`. This requires the `%qc.abs20` specifier on the operand.

```
qc.li a0, %qc.abs20(symbol+addend) # R_RISCV_VENDOR QUALCOMM
                                   # R_RISCV_QC_ABS20_U symbol, addend
```

2.3.2. Procedure Calls

Unconditional 48-bit call (*QC.EJ-Type*) instructions (in `Xqcilb`) have a `R_RISCV_QC_E_CALL_PLT` relocation that can represent an even signed 32-bit offset (-2GiB to +2GiB-2) from PC.

Just like `R_RISCV_CALL_PLT`, it is up to the linker to decide whether this relocation needs a PLT entry, or can directly use the procedure's address. Just like `R_RISCV_CALL_PLT`, calling via an instruction that uses this relocation may clobber non-callee-saved registers.

```
qc.e.jal symbol # R_RISCV_VENDOR QUALCOMM
                # R_RISCV_QC_E_CALL_PLT symbol

qc.e.j symbol  # R_RISCV_VENDOR QUALCOMM
                # R_RISCV_QC_E_CALL_PLT symbol
```

2.3.3. PC-Relative Branches

Conditional 48-bit branch (*QC.EB-type*) instructions (in `Xqcibi`) have a `R_RISCV_QC_E_BRANCH` relocation that can represent an even signed 13-bit offset (-4096 to +4094) from PC.

```
qc.e.beqi a0, 0xFF00, symbol+addend # R_RISCV_VENDOR QUALCOMM
                                     # R_RISCV_QC_E_BRANCH symbol, addend
```

Conditional branch immediate (B-type) instructions (in `Xqcibi`) use the standard `R_RISCV_BRANCH` relocation, which can represent an even signed 13-bit offset (-4096 to +4094) from PC. These use a standard relocation code, so do not need a `R_RISCV_VENDOR` relocation.

```
qc.beqi a0, 0xFF, symbol+addend # R_RISCV_BRANCH symbol, addend
```

2.4. Attributes

The RISC-V ELF psABI defines a method for providing additional information about object files that a linker or runtime loader can use to ensure the ELF files are compatible. Attributes with a tag number above 32768 are reserved for non-standard extensions.

This psABI Extension does not allocate any non-standard attribute tags.

2.5. Additional Linker Relaxation Types

The following linker relaxation types are allowed in this psABI, in addition to those defined in the RISC-V ELF psABI.

All these linker relaxations have an implicit requirement that the base architecture is RV32 (like the rest of this psABI extension). Any specific ISA Extensions required to perform the relaxation are documented with each relaxation. The relocatable object's build attributes (if present) should be used to work out if an extension is present, rather than the link result object's merged build attributes.



These relaxations are defined to be applied once, rather than the linker relaxing a sequence or group it has already relaxed. We do not define a preferential order between relaxations, it is understood that the linker performing relaxation will always choose the smallest of the results for which all the conditions are satisfied. Candidates and results below are annotated with their code size (in bytes).



In some places, we require the pair of relaxed instructions to be adjacent. This is required when a sequence of instructions is replaced with a shorter single-instruction sequence, but the replacement instruction is longer than the original instructions. The restriction is needed because we cannot always extend the length of relative references that might appear middle of the sequence.

2.5.1. Standard to Extended Function/Tail Call Relaxation

Target Relocation

`R_RISCV_CALL_PLT`, `R_RISCV_CALL`

Description

This relaxation type can relax `AUIPC+JALR` into `QC.E.J` or `QC.E.JAL`

Conditions

- The offset between the relocated location and the target symbol or the PLT stub of the target symbol is within range of $\pm 2\text{GiB}$; and
- The `rd` of the `JALR` must be `ra` or `x0`.

Extensions Required

`Xqci1b`

Relaxation

Instruction sequence associated with `R_RISCV_CALL_PLT` (or `R_RISCV_CALL`) can be rewritten into a single `QC.E.JAL` or `QC.E.J` (depending on the destination register of the `JALR`).



Using address of PLT stubs of the target symbol or address target symbol directly will be resolved by linker according to the visibility of the target symbol.

Example

Relaxation candidate:

```
auipc ra, 0      # R_RISCV_CALL_PLT symbol
                # R_RISCV_RELAX
jalr  ra, ra, 0
# 8 bytes
```

Relaxation result:

```
qc.e.jal 0      # R_RISCV_VENDOR QUALCOMM
                # R_RISCV_QC_E_CALL_PLT symbol
# 6 bytes
```

This can also be used for tail calls, where the destination register of **JALR** is **X0**.

Relaxation candidate:

```
auipc ra, 0      # R_RISCV_CALL_PLT symbol
                # R_RISCV_RELAX
jalr  x0, ra, 0
# 8 bytes
```

Relaxation result:

```
qc.e.j 0        # R_RISCV_VENDOR QUALCOMM
                # R_RISCV_QC_E_CALL_PLT symbol
# 6 bytes
```

2.5.2. Extended to Standard Function/Tail Call Relaxation

Target Relocation

`R_RISCV_QC_E_CALL_PLT`

Description

This relaxation can relax **QC.E.J** or **QC.E.JAL** into **JAL**.

Condition

The offset between the relocated location and the target symbol or the PLT stub of the target symbol is within range of $\pm 1\text{MiB}$.

Relaxation

Instruction associated with `R_RISCV_QC_E_CALL_PLT` can be rewritten into a single **JAL** instruction.

Example

Relaxation candidate:

```
qc.e.jal 0 # R_RISCV_VENDOR QUALCOMM
          # R_RISCV_QC_E_CALL_PLT symbol
          # R_RISCV_RELAX
# 6 bytes
```

Relaxation result:

```
jal ra, 0 # R_RISCV_JAL symbol
# 4 bytes
```

Relaxation candidate:

```
qc.e.j 0 # R_RISCV_VENDOR QUALCOMM
          # R_RISCV_QC_E_CALL_PLT symbol
          # R_RISCV_RELAX
# 6 bytes
```

Relaxation result:

```
jal x0, 0 # R_RISCV_JAL symbol
# 4 bytes
```

2.5.3. Extended to Compressed Function/Tail Call Relaxation

Target Relocation

R_RISCV_QC_E_CALL_PLT

Description

This relaxation can relax **QC.E.JAL** or **QC.E.J** into **C.JAL** or **C.J** respectively.

Condition

The offset between the relocated location and the target symbol or the PLT stub of the target symbol is within range of $\pm 2\text{KiB}$.

Extensions Required

Zca

Relaxation

QC.E.JAL associated with **R_RISCV_QC_E_CALL_PLT** can be rewritten into a single **C.JAL** instruction.

Example

Relaxation candidate:

```
qc.e.jal 0    # R_RISCV_VENDOR QUALCOMM
              # R_RISCV_QC_E_CALL_PLT symbol
              # R_RISCV_RELAX
# 6 bytes
```

Relaxation result:

```
c.jal 0      # R_RISCV_RVC_JUMP symbol
# 2 bytes
```

Relaxation candidate:

```
qc.e.j 0     # R_RISCV_VENDOR QUALCOMM
              # R_RISCV_QC_E_CALL_PLT symbol
              # R_RISCV_RELAX
# 6 bytes
```

Relaxation result:

```
c.j 0        # R_RISCV_RVC_JUMP symbol
# 2 bytes
```

2.5.4. Standard to Extended Global-pointer Relaxation

Target Relocation

R_RISCV_HI20, R_RISCV_LO12_I, R_RISCV_LO12_S, R_RISCV_PCREL_HI20,
R_RISCV_PCREL_LO12_I, R_RISCV_PCREL_LO12_S

Description

This relaxation type can relax a sequence of a load or store with a symbol reference into a global-pointer-relative load or store using **qc.e.l*** or **qc.e.s*** instructions respectively.

Condition

- The offset between the global pointer and the symbol is within range of $\pm 32\text{MiB}$; and
- The **lui** and the load or store are adjacent instructions.

Extensions Required

Xqcilo

Relaxation

- Instruction associated with R_RISCV_HI20 or R_RISCV_PCREL_HI20 can be removed.
- Instruction associated with R_RISCV_LO12_I, R_RISCV_LO12_S, R_RISCV_PCREL_LO12_I or R_RISCV_PCREL_LO12_S can be replaced with a global-pointer-relative access instruction.

Load Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
lw t1, 0(t0)   # R_RISCV_LO12_I symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.e.lw t1, <gp-offset-for-symbol>(gp)
# 6 bytes
```

Store Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
sw t1, 0(t0)   # R_RISCV_LO12_S symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.e.sw t1, <gp-offset-for-symbol>(gp)
# 6 bytes
```

2.5.5. Standard to Extended Global-pointer Address Relaxation

Target Relocation

R_RISCV_HI20, R_RISCV_LO12_I, R_RISCV_PCREL_HI20, R_RISCV_PCREL_LO12_I

Description

This relaxation type can relax a sequence for loading the address of a symbol reference into a global-pointer-relative **qc.e.addi** instruction.

Conditions

- The offset between the global pointer and the symbol is within range of $\pm 32\text{MiB}$; and
- The **lui** and the **addi** are adjacent instructions.

Extensions Required

Xqcilia

Relaxation

- Instruction associated with `R_RISCV_HI20` or `R_RISCV_PCREL_HI20` can be removed.
- Instruction associated with `R_RISCV_L012_I` or `R_RISCV_PCREL_L012_I` can be replaced with a global-pointer-relative `QC.E.ADDI` instruction.

Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
addi t1, t0, 0 # R_RISCV_L012_I symbol
               # R_RISCV_RELAX

# 8 bytes
```

Relaxation result:

```
qc.e.addi t1, gp, <gp-offset-for-symbol>

# 6 bytes
```

2.5.6. Extended to Standard Global-pointer Address Relaxation

Target Relocation

`R_RISCV_QC_E_32`

Description

This relaxation type can relax a `qc.e.li` instruction for loading the address of a symbol reference into an `addi` instruction.

Condition

The offset between the global pointer and the symbol is within range of $\pm 2\text{KiB}$.

Relaxation

Instruction associated with `R_RISCV_QC_E_32` can be replaced with an `ADDI` instruction.

Example

Relaxation candidate:

```
qc.e.li t1, 0 # R_RISCV_VENDOR QUALCOMM
             # R_RISCV_QC_E_32 symbol
             # R_RISCV_RELAX

# 6 bytes
```

Relaxation result:

```
addi t1, gp, <gp-offset-for-symbol>

# 4 bytes
```

2.5.7. Standard to Extended Zero-page Load/Store Relaxation

Target Relocation

R_RISCV_HI20, R_RISCV_LO12_I, R_RISCV_LO12_S, R_RISCV_PCREL_HI20,
R_RISCV_PCREL_LO12_I, R_RISCV_PCREL_LO12_S

Description

This relaxation type can relax a sequence of a load or store with a symbol reference into an absolute load or store using `qc.e.l*` or `qc.e.s*` instruction respectively.

Conditions

- The symbol address is located within `0x0 ~ 0x03ffffff` or `0xfc000000 ~ 0xffffffff`; and
- The `lui` and the load or store are adjacent instructions.

Extensions Required

Xqcilo

Relaxation

- Instruction associated with `R_RISCV_HI20` or `R_RISCV_PCREL_HI20` can be removed.
- Instruction associated with `R_RISCV_LO12_I`, `R_RISCV_LO12_S`, `R_RISCV_PCREL_LO12_I` or `R_RISCV_PCREL_LO12_S` can be replaced with an access instruction relative to `zero`.

Load Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
lw t1, 0(t0)   # R_RISCV_LO12_I symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.e.lw t1, <address-of-symbol>(zero)
# 6 bytes
```

Store Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
sw t1, 0(t0)   # R_RISCV_LO12_S symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.e.sw t1, <address-of-symbol>(zero)
# 6 bytes
```

2.5.8. Standard to Extended Address Relaxation

Target Relocation

R_RISCV_HI20, R_RISCV_LO12_I, R_RISCV_PCREL_HI20, R_RISCV_PCREL_LO12_I

Description

This relaxation type can relax a sequence for loading the address of a symbol reference into an absolute `qc.e.li` instruction.

Extensions Required

Xqcili

Condition

The `lui` and the `addi` are adjacent instructions.

Relaxation

- Instruction associated with R_RISCV_HI20 or R_RISCV_PCREL_HI20 can be removed.
- Instruction associated with R_RISCV_LO12_I or R_RISCV_PCREL_LO12_I can be replaced with a

QC.E.LI instruction.

Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
addi t1, 0(t0) # R_RISCV_L012_I symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.e.li t1, <address-of-symbol>
# 6 bytes
```

2.5.9. Zero-page Address Relaxation

Target Relocation

R_RISCV_HI20, R_RISCV_LO12_I, R_RISCV_PCREL_HI20, R_RISCV_PCREL_LO12_I

Description

This relaxation type can relax a sequence for loading the address of a symbol reference into an absolute `qc.li` instruction.

Condition

The symbol address is located within `0x0 ~ 0x000fffff` or `0xffff0000 ~ 0xffffffff`.

Extensions Required

Xqcili

Relaxation

- Instruction associated with R_RISCV_HI20 or R_RISCV_PCREL_HI20 can be removed.
- Instruction associated with R_RISCV_LO12_I or R_RISCV_PCREL_LO12_I can be replaced with a QC.LI instruction.

Example

Relaxation candidate:

```
lui t0, 0      # R_RISCV_HI20 symbol
               # R_RISCV_RELAX
addi t1, 0(t0) # R_RISCV_L012_I symbol
               # R_RISCV_RELAX
# 8 bytes
```

Relaxation result:

```
qc.li t1, <address-of-symbol>
# 4 bytes
```

2.5.10. Extended to Standard Address Relaxation

Target Relocation

R_RISCV_QC_E_32

Description

This relaxation type can relax a **qc.e.li** instruction for loading the address of a symbol reference into a **qc.li** instruction.

Condition

The symbol address is located within **0x0 ~ 0x000fffff** or **0xffff0000 ~ 0xffffffff**.

Extensions Required

Xqcili



In this Relaxation, both the candidate instruction and the result instruction are from the same extension, so this can be performed even when architecture attributes are not available.

Relaxation

Instruction associated with **R_RISCV_QC_E_32** can be replaced with a **QC.LI** instruction.

Example

Relaxation candidate:

```
qc.e.li t1, 0 # R_RISCV_VENDOR QUALCOMM
              # R_RISCV_QC_E_32 symbol
              # R_RISCV_RELAX

# 6 bytes
```

Relaxation result:

```
qc.li t1, <address-of-symbol>
# 4 bytes
```



*This relaxation could have used **ADDI** as the destination instruction (which is also 4 bytes), but **QC.LI** is valid everywhere **QC.E.LI** is valid, and has a larger immediate range than **ADDI**, which is why we chose **QC.LI** instead.*

2.5.11. Extended to Compressed Address Relaxation

Target Relocation

R_RISCV_QC_E_32, R_RISCV_QC_ABS20_U

Description

This relaxation type can relax a **qc.e.li** or **qc.li** instruction for loading the address of a symbol reference into a **c.li**.

Condition

The symbol address is located within **0x0 ~ 0x3f** or **0xffffffffc0 ~ 0xffffffffff**.

Extensions Required

Zca

Relaxation

Instruction associated with R_RISCV_QC_E_32 or R_RISCV_QC_ABS20_U can be replaced with a **C.LI** instruction.

Example

Relaxation candidate:

```
qc.e.li t1, 0    # R_RISCV_VENDOR QUALCOMM  
                # R_RISCV_QC_E_32 symbol  
                # R_RISCV_RELAX  
  
# 6 bytes
```

Relaxation result:

```
c.li t1, <address-of-symbol>  
# 2 bytes
```

Relaxation candidate:

```
qc.li t1, 0      # R_RISCV_VENDOR QUALCOMM  
                # R_RISCV_QC_ABS20_U symbol  
                # R_RISCV_RELAX  
  
# 4 bytes
```

Relaxation result:

```
c.li t1, <address-of-symbol>  
# 2 bytes
```

Appendix A: Commentary

This section seeks to answer some common questions about the direction we took in designing the psABI extension, for clarity and for future reference.

A.1. Why Have We Not Reused Standard Relocations?

We have where we could, for example for the 32-bit branch-immediate instructions in Xqcibi, which use `R_RISCV_BRANCH` relocations.

We cannot for the 48-bit instructions (even though it looks like we might be able to, at first glance), for a few reasons:

- QC.EB-type instructions (such as `qc.e.beqi`) where the PC-relative offset is laid out in fields that exactly match the standard conditional branch instruction (i.e. `beq`). Note these instructions have both an `sim` field — which is sign-extended and compared — and an `imm` field - which will be added to the PC to calculate where to branch to. We cannot use `R_RISCV_BRANCH` relocations to fill in the PC-relative offset (though the bits being written by the relocation are identical), because the linker relaxation process needs to know which bytes are part of the relocated instruction, rather than just the location of the fields it might be updating. Therefore we add a new relocation for these instructions: `R_RISCV_QC_E_BRANCH`.
- QC.EAI-type instructions (such as `qc.e.li`) where the 32-bit immediate is laid out in a contiguous field from bits 16-47 of the instruction. We cannot use `R_RISCV_32` relocations here, because this is a data relocation, and on big endian systems instructions remain little-endian. Using this relocation would also cause us the same issues with linker relaxation, as we would not be able to easily work out the instruction extents. Therefore we add a new relocation for use with `qc.e.li`: `R_RISCV_QC_E_32`.

A.2. Why Is There Not A Version of the `R_RISCV_QC_E_CALL_PLT` Relocation Without The PLT?

We see the `qc.e.jal` and `qc.e.j` instructions as the equivalent of `call` and `tail`, in a single 48-bit instruction. RISC-V originally had two call relocations, one for via the PLT and a direct version. Some time later it was understood that the non-PLT version is unnecessary, as linkers are allowed to omit the PLT entry in certain circumstances (including when static linking) and just resolve directly to the symbol in question.

We sought to avoid the same pitfalls of the main RISC-V psABI, so defined only a PLT version of the relocation, which linkers can resolve directly to the relevant function where appropriate.

A.3. Why Have We Not Defined a New Code Model?

The existing RV32 code models focus on addressing the entire 32-bit address space — either using absolute address materialization, or using PC-relative address materialization (potentially via a GOT).

Embedded systems tend to have lower requirements on the size of their address spaces and also want to use shorter code sequences for address materialization within these space constraints (e.g., only using 20-bit addresses).

There are two reasons we have not defined and implemented a code model.

The first reason is it requires a lot of implementation and maintenance across the toolchain (compilers, linkers, libraries), which we do not believe is justified at this time, especially as this code model will be

harder to use than the conventional 32-bit models.

The second reason is that there is no clear consensus on what size this smaller address space should be within the instructions defined in the RV32 or Xqci specifications.

Here are some relevant address materialization options found within RV32 and Xqci, glossing over that some of these types also have alignment requirements on their immediates:

- Signed 6-bit absolute (RVC CI-type instructions using `x0`)
- Signed 12-bit absolute (RV32 I-type and S-type instructions using `x0`)
- Signed 18-bit absolute (RV32 I-type and S-type with `c.lui`)
- Signed 20-bit absolute (Xqci `qc.li`)
- Signed 32-bit absolute (Xqci `qc.e.li`)
- Signed 9-bit relative to PC (RVC CB-type)
- Signed 12-bit relative to PC (RVC CJ-type)
- Signed 13-bit relative to PC (RV32 B-type and Xqci QC.EB-type)
- Signed 21-bit relative to PC (RV32 J-type)
- Signed 32-bit relative to PC (Xqci QC.EJ-type)
- Signed 6-bit relative to `gp` (RVC CI-type)
- Signed 12-bit relative to `gp` (RV32 I-type and S-type)
- Signed 26-bit relative to `gp` (Xqci QC.EI-type and QC.ES-type)
- Signed 6-bit relative to `sp` (RVC CI-type)
- Signed 12-bit relative to `sp` (RV32 I-type and S-type)
- Signed 26-bit relative to `sp` (Xqci QC.EI-type and QC.ES-type)
- Signed 10-bit relative to `sp` (RVC `c.addi16sp`)
- Unsigned 10-bit relative to `sp` (RVC `c.addi4spn`)

From this, it is clear there is no consensus over what size of address space lower than 32-bits would actually be useful enough in general for embedded projects for support in open source toolchains. Every code model has a maintenance overhead, so it is not possible or sustainable for open source toolchains to add a new code model for specific embedded projects.

A.4. Why Are There No Relocations for the 48-bit Load and Store Instructions?

There are a complex series of reasons behind these relating to code models, materialization sequences and linker relaxations.

Recall that the `Xqcilo` instructions have the following form, where the address that is loaded or stored is within 26-bits of the value in register `rs1`:

```
qc.e.lw rd, simm(rs1)
```

There are several options for using this to materialize an address.

One is to use this to materialize a full 32-bit address, as the **medlow** and **medany** code models require. There is no instruction to materialize the remaining 6 bits of the address, so we would need to use **lui** or **auipc** to load the top bits into **rs1**. All sequences including **lui** or **auipc** and an **Xqcilo** load or store are equivalent to a shorter code sequence of the **lui** or **auipc** followed by a standard load or store, so these sequences do not make sense. If there was a 16-bit instruction for materialising the top 6 bits of the address, the code sequence would still be 8 bytes long, as it is with the conventional sequence.

The reasons for not defining a new code model suitable are described in [Section A.3](#).

Another option would be to only allow **rs1** to be **x0**, and materialize an absolute 26-bit signed address. This model of relocation, with specific register constraints would be new in the psABI. This similarly applies to a GP-relative address materialization that would require **rs1** to be **gp**. While GP-relative relocations are not per-se new, any that were defined in the past have been removed, as they were a binutils linker implementation detail, and since then no relocations with register-specific constraints have been added.

This leaves a final option: linker relaxation, which is the direction we have chosen to take. With this option, compilers and assemblers emit the existing, standard materialisation sequence for load or store consisting of an **lui** or **auipc** followed by a load or store, and if the circumstances are right, we can relax it to either a **x0** or **gp** relative **Xqcilo** load or store.

In order to support the linker relaxation in the assembler, we have added pseudo-instruction versions of the **Xqcilo** loads and stores that take a symbol expression (akin to the standard load and store pseudo instructions that take a symbol). These emit the standard materialization sequences for the loads and stores, with the expectation that if relaxation is enabled they may be turned into **Xqcilo** loads and stores. For example:

```
qc.e.lw a0, symbol+addend

# expands to:
.Lpcrel_hi1:
auipc a0, %pcrel_hi(symbol+addend) # R_RISCV_RELAX
lw a0, %pcrel_lo(.Lpcrel_hi1)(a0)  # R_RISCV_RELAX

# which may be relaxed to:
qc.e.lw a0, symbol+addend(x0)
```

A.5. Why Do Some Relaxations Require Adjacent Instructions?

The short answer is that the instruction boundaries are changing, which has complex effects.

In particular, sequences where there are branches between the two instructions to after the final instruction can get broken, because this offset can get 2 bytes longer, even when the overall code size is smaller. There are specific circumstances where this addition can end up with the offset overflowing immediate fields, for example:

```
lui t0, %hi(symbol)
beq a0, a1, .+4094
lw a0, %lo(symbol)(t0)
# 4090 bytes of code
ret
```

```
# After relaxing the lui/lw pair to qc.e.lw, the beq becomes invalid:
beq a0, a1, .+4096 # this offset is no longer representable
qc.e.lw a0, symbol+addend(x0)
# 4090 bytes of code
ret
```

If the instructions are adjacent, this is not a problem because there cannot be any instructions in between the two instructions. This is a better solution than having the linker scan all the code between the `lui` and the `lw`, especially as there can be control flow between the two instructions.

Bibliography

Cheng, K., & Clarke, J. (Eds.). (2022). *RISC-V ABIs Specification* (Version 1.0) [Specification]. RISC-V International. github.com/riscv-non-isa/riscv-elf-psabi-doc/releases/tag/v1.0

Hower, D., & Yosher, A. (2025). *Qualcomm uC Extensions* (Version 0.13.0) [Specification]. Qualcomm Technologies, Inc. github.com/quic/riscv-unified-db/releases/tag/Xqci-0.13.0

Müllner, C., & Pazos, A. (Eds.). (2024). *RISC-V Toolchain Conventions* [Document]. RISC-V International. github.com/riscv-non-isa/riscv-toolchain-conventions